

Week 8 Lab: I2C Protocol — MPU6050 IMU

EE0827 Embedded System Applications

Overview

Interface an MPU6050 6-axis IMU with the STM32 Nucleo using I2C. You will read the WHO_AM_I register, burst-read accelerometer/gyroscope data, and capture the bus traffic with a logic analyzer.

Time allocation: 90 minutes

- Hardware & CubeMX setup: 20 min
- Basic communication (WHO_AM_I): 25 min
- Sensor reading & motion test: 25 min
- Logic analyzer & failure injection: 20 min

Learning Objectives

1. Wire an I2C device with appropriate pull-ups
2. Configure the STM32 I2C peripheral and HAL Mem_Read/Write
3. Parse multi-byte big-endian sensor data into physical units
4. Capture and decode I2C transactions with a logic analyzer

Safety & Setup

Safety Checklist

- ☐ USB cable connected to Nucleo board (USB power only)
- ☐ GY-521 VCC on **3.3V** rail (most modules accept 5V, but use 3.3V to be safe)
- ☐ **SDA and SCL not swapped** (very common mistake!)
- ☐ Logic analyzer ground connected to circuit ground

Hardware: Nucleo-F401RE, GY-521 (MPU6050) module, breadboard, jumper wires, $2 \times 4.7 \text{ k}\Omega$ resistors (only if module lacks onboard pull-ups), Saleae-clone logic analyzer.

Software: STM32CubeIDE, serial terminal (115200 baud), PulseView.

Pre-Lab Questions

1. The MPU6050 7-bit address with AD0=GND is 0x68. What 8-bit value does HAL pass on the wire for a write? For a read?

2. The WHO_AM_I register (0x75) returns what value on a healthy MPU6050? What does a genuine MPU6500 (same GY-521 footprint) return instead?

3. With $4.7 \text{ k}\Omega$ pull-ups on a 3.3 V bus, what current flows when SDA is pulled LOW?

Step 1.1: Wire the MPU6050

MPU6050 Pin	Nucleo Pin	Notes
VCC	3.3V	CN6 Pin 4
GND	GND	CN6 Pin 6 — common ground critical
SCL	PB8	I2C1_SCL (CN10 Pin 3)
SDA	PB9	I2C1_SDA (CN10 Pin 5)
AD0	GND	Sets address to 0x68
INT, XDA, XCL	(not connected)	Unused

Logic analyzer: CH0 → SCL (PB8), CH1 → SDA (PB9), GND → board GND.

Pull-ups: external or onboard?

Most GY-521 boards include onboard $2.2\text{k}\Omega$ or $4.7\text{k}\Omega$ pull-ups, but they sit on the output of the board's 3.3 V LDO, **not the VCC pin**. An ohmmeter between VCC and SDA reads through the LDO in series and will show hundreds of $\text{k}\Omega$ to megohms — that is *not* a pull-up failure. The correct check is done with power applied: verify SDA and SCL idle near 3.3 V. Adding external $4.7\text{k}\Omega$ in parallel is always safe.

Pre-power check (multimeter, board unpowered):

1. Continuity: no short between VCC and GND

Powered check (3.3 V applied, bus idle):

2. SDA to GND reads 3.3 V (pull-up working)
3. SCL to GND reads 3.3 V (pull-up working)

Step 1.2: CubeMX Project

1. **New project** → Board: NUCLEO-F401RE → Name: W08_I2C_IMU
2. **Connectivity** → **I2C1**: Mode = I2C, verify PB8/PB9 selected
 - Speed Mode: Standard, Clock Speed: 100000 (100 kHz), Addressing: 7-bit
3. **Connectivity** → **USART2**: Asynchronous, 115200 baud, 8N1
4. **Generate code** and open in CubeIDE.

Step 2.1: Defines and printf Redirect

Inside `main.c`:

```
/* USER CODE BEGIN Includes */
#include <stdio.h>
/* USER CODE END Includes */

/* USER CODE BEGIN PD */
#define MPU6050_ADDR      (0x68 << 1)
#define REG_WHO_AM_I      0x75
#define REG_PWR_MGMT_1     0x6B
#define REG_ACCEL_XOUT_H 0x3B
/* USER CODE END PD */

/* USER CODE BEGIN 0 */
int _write(int file, char *ptr, int len) {
    HAL_UART_Transmit(&huart2, (uint8_t*)ptr, len, HAL_MAX_DELAY);
    return len;
}
/* USER CODE END 0 */
```

Keep your code inside USER CODE markers!

CubeMX **deletes** everything outside USER CODE BEGIN/END blocks when you regenerate. Always place your code inside these markers.

Step 2.2: I2C Scanner + WHO_AM_I Read

In `main()` after peripheral init:

```
/* USER CODE BEGIN 2 */
printf("\r\n=== W08: I2C IMU Lab ===\r\n");

// Scan the bus
printf("Scanning I2C bus...\r\n");
for (uint8_t a = 1; a < 128; a++) {
    if (HAL_I2C_IsDeviceReady(&hi2c1, a << 1, 1, 10) == HAL_OK)
```

```

        printf(" Found device at 0x%02X\r\n", a);
    }

    // Read WHO_AM_I
    uint8_t who = 0;
    HAL_StatusTypeDef st = HAL_I2C_Mem_Read(&hi2c1, MPU6050_ADDR,
        REG_WHO_AM_I, I2C_MEMADD_SIZE_8BIT, &who, 1, 100);
    const char *chip = (who == 0x68) ? "MPU6050"
        : (who == 0x70) ? "MPU6500"
        : "unknown";
    printf("WHO_AM_I = 0x%02X (%s) %s\r\n", who, chip,
        (st == HAL_OK && (who == 0x68 || who == 0x70)) ? "(OK)" : "(FAIL)");
    /* USER CODE END 2 */

```

Build, flash, open terminal at 115200 baud, press reset.

Expected output (MPU6050 shown — MPU6500 boards report 0x70 (MPU6500)):

```

=== W08: I2C IMU Lab ===
Scanning I2C bus...
    Found device at 0x68
WHO_AM_I = 0x68 (MPU6050) (OK)

```

GY-521 might not be an MPU6050

Many "GY-521 MPU6050" modules now ship with the newer **MPU6500** silicon, which is register-compatible for this lab but returns 0x70 from WHO_AM_I. That's why our check accepts both 0x68 and 0x70. Other possible IDs: 0x71 (MPU9250), 0x73 (MPU9255), 0x98 (ICM-20689).

Checkpoint 1

You must see `Found device at 0x68` and a WHO_AM_I of 0x68 or 0x70 marked (OK) before continuing. If not, see **Common Issues** on the last page.

Step 3.1: Wake the MPU6050

The device boots in sleep mode. Add **after** the WHO_AM_I check:

```
uint8_t pwr = 0x00;    // clear SLEEP bit
HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR, REG_PWR_MGMT_1,
                  I2C_MEMADD_SIZE_8BIT, &pwr, 1, 100);
HAL_Delay(100);
```

Step 3.2: Burst-Read 14 Bytes

```
/* USER CODE BEGIN PD */
#define ACCEL_LSB      16384.0f    // ±2 g range
#define GYRO_LSB       131.0f     // ±250 °/s range
// Temperature formula differs between chip variants:
//   MPU6050: T_C = T_raw/340      + 36.53
//   MPU6500: T_C = T_raw/333.87   + 21.00
#define TEMP_DIV_6050   340.00f
#define TEMP_OFS_6050   36.53f
#define TEMP_DIV_6500   333.87f
#define TEMP_OFS_6500   21.00f
/* USER CODE END PD */
```

In the main `while(1)` loop. Use the `who` value read in Part 2 to pick the right temperature formula:

```
/* USER CODE BEGIN WHILE */
uint8_t buf[14];
int16_t ax_r, ay_r, az_r, t_r, gx_r, gy_r, gz_r;

while (1) {
    if (HAL_I2C_Mem_Read(&hi2c1, MPU6050_ADDR, REG_ACCEL_XOUT_H,
                        I2C_MEMADD_SIZE_8BIT, buf, 14, 100) == HAL_OK) {
        ax_r = (buf[0] << 8) | buf[1];
        ay_r = (buf[2] << 8) | buf[3];
        az_r = (buf[4] << 8) | buf[5];
```

```

t_r  = (buf[6]  << 8) | buf[7];
gx_r = (buf[8]  << 8) | buf[9];
gy_r = (buf[10] << 8) | buf[11];
gz_r = (buf[12] << 8) | buf[13];

float temp_c = (who == 0x70)
    ? (t_r / TEMP_DIV_6500) + TEMP_OFS_6500 // MPU6500
    : (t_r / TEMP_DIV_6050) + TEMP_OFS_6050; // MPU6050

printf("A: %+5.2f %+5.2f %+5.2f g | "
      "G: %+6.1f %+6.1f %+6.1f dps | %.1fC\r\n",
      ax_r/ACCEL_LSB, ay_r/ACCEL_LSB, az_r/ACCEL_LSB,
      gx_r/GYRO_LSB,  gy_r/GYRO_LSB,  gz_r/GYRO_LSB,
      temp_c);
}
HAL_Delay(100); // 10 Hz
/* USER CODE END WHILE */

```

Why does the temperature formula differ?

The MPU6050 and MPU6500 use *different* temperature sensor slopes and offsets in their datasheets:

$$T_{6050} = \frac{T_{\text{raw}}}{340} + 36.53 \qquad T_{6500} = \frac{T_{\text{raw}}}{333.87} + 21.00$$

Using the wrong formula shifts the reading by roughly 15°C. At room temperature the die sits 5–10°C above ambient from self-heating, so a healthy reading should be around 27–33°C.

Note: Declare `who` at file scope (or pass it in) if your sensor-read loop can't see the local from `main()`.

Step 3.3: Motion Tests

Test	Expected	Measured
Flat on bench	Az +1.0 g, Ax/Ay 0	
Tilt forward	Ax increases	
Tilt left	Ay increases	
Quick Z rotation	Gz spikes ±50 to ±200 °/s	
Flat idle temperature	27–33 °C (5–10 °C above ambient)	
Touch chip	Temp rises above ambient	

Why does Az read +1g when flat?

The accelerometer measures the *normal force* from the bench, not gravity directly. Free-falling, all axes read zero.

Checkpoint 2

Sensor responds to tilt and rotation, temperature is in 20–35°C range.

Step 4.1: Capture WHO_AM_I

1. PulseView → Sample rate **2 MHz**, 100 ms capture
2. Add **I2C** decoder: SCL = D0, SDA = D1
3. Start capture, press reset on Nucleo, stop capture

Identify on the trace: START → 0xD0 (write) → ACK → 0x75 → ACK → Sr → 0xD1 (read) → ACK → 0x68 → NACK → STOP.

Step 4.2: Capture the 14-Byte Burst Read

Start a new 200 ms capture, wait one sensor cycle, stop.

Measurement	Expected	Measured
SCL frequency	~100 kHz	_____ kHz
Single byte time	~90 μ s	_____ μ s
Full 14-byte read	~1.5 ms	_____ ms
SDA rise time	< 1 μ s	_____ ns

Save annotated screenshots of both captures.

Step 4.3: Failure Injection

For each test: **predict**, **test**, **restore**.

Test 1 — Wrong address. Change MPU6050_ADDR to (0x69 << 1), rebuild, flash. **Predict:** ACK NACK. **Observed:** _____

Test 2 — Disconnect SDA briefly while running. **Predict** error behavior. **Observed:** ****Restore.****

Test 3 — Add 47 k Ω in parallel with pull-ups (or remove one if external). Watch SDA rise time on the analyzer. **Observed:** _____ ****Restore.****

Measurement Summary

ID	Measurement	Expected	Measured	Pass
M1	SDA idle voltage (powered)	~3.3 V		
M2	SCL idle voltage (powered)	~3.3 V		
M3	I2C scan finds	0x68		
M4	WHO_AM_I value	0x68 or 0x70		
M5	Az flat on bench	~+1.0 g		
M6	SCL frequency	~100 kHz		
M7	14-byte burst time	~1.5 ms		

Analysis Questions

1. Why does I2C use open-drain outputs with pull-ups, while SPI uses push-pull?

2. We read 14 bytes in one burst. Roughly how much faster is this than 7 separate 2-byte reads, and why?

3. What would happen with 47 k Ω pull-ups instead of 4.7 k Ω ? With 470 Ω ?

Deliverables

Completion Checklist

- ☐ WHO_AM_I read returns 0x68
- ☐ Sensor reading responds to tilt/rotation
- ☐ Annotated logic-analyzer screenshots (WHO_AM_I + burst read)
- ☐ Measurement table completed
- ☐ Failure injection tests documented
- ☐ Analysis questions answered

This lab is **ungraded practice**.

Common Issues

Issue	Fix
Scan finds no device	Check 3.3 V on VCC, common GND, SDA/SCL not swapped
WHO_AM_I = 0xFF	Pull-ups missing or too weak — with power on, SDA should idle near 3.3 V (do not use an ohmmeter from VCC, the onboard LDO blocks it)
WHO_AM_I = 0x70	Board is an MPU6500, not an MPU6050 — accept it as valid (register-compatible)
WHO_AM_I = 0x71 / 0x73	Board is an MPU9250 / MPU9255 — accel + gyro registers are identical, safe to use
WHO_AM_I = 0x00	Module in reset; check VCC, try power cycle
Sensor data garbage	Forgot wake-up write to PWR_MGMT_1, or wrong byte order
Logic analyzer blank	Sample rate must be 1 MHz; share GND with circuit
Bus stuck (SDA LOW)	Power-cycle the module

Key Takeaways

1. **HAL shifts the address** — pass $(0x68 \ll 1)$, not 0x68
2. **Burst read is essential** — one START/STOP per 14 bytes beats 7 separate transactions
3. **Pull-ups define rise time** — too weak = rounded edges, too strong = wasted current
4. **Always WHO_AM_I first** — it's the cheapest sanity check before debugging sensor parsing